

Motion Detection Application: Development Plan

Version 1.0 · Windows desktop · Python, OpenCV, Tkinter · fully offline

1. Project overview

The Motion Detection Application is a desktop application that monitors movement through a connected webcam in real time. It accesses the client's camera, analyzes the live video feed continuously, and shows an alert such as "Motion Detected!" directly on the camera feed whenever movement is found.

The application does not record, save or store any video footage or captured images. All processing happens locally on the client's computer, with no internet connection required.

This plan covers how the application will be built, tested and delivered. It follows the System Architecture document (version 1.0): the worker-thread and queue design, the MOG2 detection pipeline, the alert state logic and the privacy-by-design rules all carry over unchanged.

2. Objectives and success criteria

Objective	Success criterion
Access the webcam and display live video	Live feed appears within 3 seconds of clicking Start Monitoring and runs smoothly at 15 FPS or better.
Detect movement in real time	A person walking into frame is flagged within about half a second.
Show an immediate visual alert	"Motion Detected!" overlay appears on the feed and the status indicator turns red.
Simple, user-friendly interface	A non-technical user can start and stop monitoring without instructions.
No recording or storage of video or images	No image or video file is created anywhere during or after a run (verified with a file monitor).
Fully local processing	The application makes no network connections (verified during testing).

3. Scope

In scope (version 1): webcam access, live display, motion detection with false-alert reduction, on-feed alert overlay, status indicator, Start and Stop buttons, camera error handling, Windows executable, source code and user instructions.

Out of scope (version 1): recording or screenshots, remote notifications (email, SMS), multiple cameras, user accounts, installers with auto-update. Sound alerts and adjustable sensitivity are listed as optional extensions in section 15.

4. Technology stack

Technology	Purpose	Notes
Python 3.10+	Main programming language	Single version fixed for development and packaging.
OpenCV (opencv-python)	Webcam access and motion detection	MOG2 background subtractor, contour analysis. DirectShow backend on Windows.
NumPy	Image and frame processing	Installed with OpenCV.
Tkinter	Desktop GUI	Ships with standard Python on Windows.
Pillow	Displaying camera frames in Tkinter	Converts frames to ImageTk.PhotoImage.
VS Code	Development environment	With the Python extension.
PyInstaller	Packaging into an executable	--onefile --windowed.
pytest, Git	Unit testing, version control	Added to support the testing approach in section 10.

5. Development phases

Work is split into eight phases. Each phase has defined tasks, an expected output and acceptance criteria. A phase is finished only when its acceptance criteria are met and the result is committed to version control.

Phase 1: Environment setup

Estimate: 0.5 day · Depends on: none

Tasks

- Install Python 3.10+ and VS Code with the Python extension.
- Create the project directory and a virtual environment.
- Install the libraries and record them in `requirements.txt`: `opencv-python`, `numpy`, `pillow` (and `pytest`, `pyinstaller` for development).
- Initialise a Git repository with a `.gitignore`.
- Write a short script that opens the webcam with OpenCV, reads one frame, prints its size and releases the camera.

Expected output: working development environment.

Acceptance criteria: the check script prints the frame size without errors; `pip install -r requirements.txt` works on a clean virtual environment.

Phase 2: Camera integration

Estimate: 0.5 to 1 day · Depends on: Phase 1 · File: `camera.py`

Tasks

- Build a camera class that opens the device (DirectShow first, default backend as fallback), reads frames and releases the camera.
- Define clear error types: no camera found, access denied or camera in use, camera disconnected during use.
- Treat several consecutive failed reads as a disconnection, not a one-off glitch.
- Show the feed in a temporary OpenCV window to confirm everything works before the GUI exists.
- Make sure the camera is always released, including when an exception occurs.

Expected output: live webcam feed.

Acceptance criteria: feed displays; each error type produces a distinct, readable message; the camera light turns off after release.

Phase 3: Motion detection implementation

Estimate: 1 to 2 days · Depends on: Phase 2 · File: `motion_detector.py`

Tasks

- Implement the pipeline: resize to 640 px wide, grayscale, Gaussian blur, MOG2 background subtraction, threshold, erode and dilate, find contours.
- Return a result per frame: motion present (true or false), total motion area and bounding boxes.
- Apply a minimum contour area to ignore tiny changes.
- Add a warm-up period (about 30 frames) during which detections are ignored while the background model learns.
- Keep every tunable value in one place (see section 8).
- Test with a person moving, no movement, a shadow, and a light being switched on or off.

Expected output: working motion detection.

Acceptance criteria: movement is detected in normal room light; a still scene produces no detections; a lighting change does not trigger repeated false detections after warm-up.

Phase 4: Alert system

Estimate: 0.5 day · Depends on: Phase 3 · File: `alert_system.py`

Tasks

- Build the alert state logic: motion must be present for 3 consecutive frames before the state becomes Alert.
- Keep the alert active for a cooldown of about 2 seconds after the last movement, so it does not flicker.
- Produce the status text: "Motion Detected!" or "No Motion Detected".
- Draw the alert overlay (banner and optional boxes) onto the frame in memory.
- Expose an `on_alert` hook so a sound or other action can be added later without touching detection code.
- Write unit tests with scripted sequences of motion values.

Expected output: real-time motion alerts.

Acceptance criteria: unit tests pass; during continuous movement the banner stays steady; single noisy frames do not raise an alert.

Phase 5: GUI development

Estimate: 1 to 2 days · Depends on: Phases 2 and 4 · File: `gui.py`

Tasks

- Build the main window: title, live camera area, status text, status indicator (grey, green, red), Start and Stop buttons, and an error message area.
- Run capture and detection in a worker thread. Pass the newest frame to the interface through a queue of size 1, and update the interface by polling every 15 to 30 ms. Only the main thread touches Tkinter widgets.
- Enable and disable the buttons according to application state, so Start cannot be pressed twice.
- Show the status text in the interface and the alert overlay on the video.
- Handle closing the window while monitoring is running.

Expected output: complete desktop interface.

Acceptance criteria: the interface stays responsive while video is processed; Start and Stop work repeatedly; error messages appear in the window, not the console.

Phase 6: Integration and privacy

Estimate: 0.5 day · Depends on: Phases 2 to 5 · File: `main.py`

Tasks

- Connect camera, detector, alert system and GUI in `main.py`.
- Search the code for `imwrite`, `VideoWriter` and any other file or network use that touches image data. None may exist.
- Check that logging contains events and errors only, never image data.
- Confirm the stop flow: set stop event, join worker, release camera, return to Idle.
- Confirm the camera is released when the window is closed.

Expected output: fully integrated application.

Acceptance criteria: the code review for privacy is clean; a full start, alert, stop cycle works end to end.

Phase 7: Testing and debugging

Estimate: 1 to 2 days · Depends on: Phase 6

Tasks

- Run the full test plan in section 10, including the privacy and network checks.
- Test different lighting conditions, distances and types of movement.
- Run repeated start and stop cycles (at least 50) and watch for memory growth or a camera that stays locked.
- Run a long session (one hour or more) while watching CPU and memory use.
- Tune the detection parameters and record the final values.
- Log every defect, fix it and retest.

Expected output: tested and stable application.

Acceptance criteria: no open critical or major defects; every test in section 10 passes.

Phase 8: Packaging and delivery

Estimate: 0.5 to 1 day · Depends on: Phase 7

Tasks

- Package with PyInstaller (`--onefile --windowed`) and add an application icon from `assets/icons/`.
- Test the `.exe` on a machine with no Python installed, ideally the client's or one similar to it.
- Re-run the main tests against the executable, including the camera error cases.
- Write the README and a short user guide with screenshots.
- Deliver the executable, source code, requirements file and documentation to the client, and walk them through the application.

Expected output: client-ready application.

Acceptance criteria: the executable runs on a clean Windows machine; the client confirms it works with their webcam.

6. Project structure

Separate files are kept for separate responsibilities, instead of one large script.

```
Motion_Detection_App/
├── main.py           # entry point, wires everything together
├── camera.py         # webcam open, read, release, errors
├── motion_detector.py # detection pipeline
├── alert_system.py   # alert state, status text, overlay
├── gui.py            # Tkinter interface and worker/queue control
├── config.py         # all tunable parameters (addition)
├── requirements.txt
├── README.md
├── tests/            # unit tests (addition)
│   ├── test_alert_system.py
│   └── test_motion_detector.py
└── assets/
    └── icons/
```

File	Responsibility	Architecture component
main.py	Application entry point	Startup, wiring
camera.py	Webcam initialization and frame capture	CameraService
motion_detector.py	Motion detection algorithm	MotionDetector
alert_system.py	Motion alert and status management	AlertManager, OverlayRenderer
gui.py	Desktop interface and monitoring control	MainWindow, VideoPanel, StatusBar, MonitorController, AppState
config.py	Tunable parameters in one place	Configuration
requirements.txt, README.md	Dependencies and documentation	n/a
assets/	Icons and visual resources	n/a

The plan keeps the architecture's modules in five files. If `gui.py` grows too large, the monitoring control can be moved to its own `controller.py` without changing anything else.

7. Motion detection approach

Detection uses OpenCV background subtraction with the MOG2 subtractor. It models the background over time, adapts to gradual lighting changes and can flag shadows so they can be ignored. Per frame:

1. Capture a frame and resize it to about 640 px wide.
2. Convert to grayscale and apply a Gaussian blur to reduce sensor noise.
3. Compare against the estimated background to produce a foreground mask.
4. Threshold the mask (which also removes shadow pixels) and clean it with erode and dilate.
5. Find contours and ignore any smaller than the minimum area.
6. Report motion if at least one contour remains; the alert logic then requires 3 consecutive frames.

8. Detection parameters

These starting values live in `config.py` and are tuned in Phase 7 using the client's real environment.

Parameter	Starting value	Effect
Processing width	640 px	Lower is faster but less sensitive to small movement.
Blur kernel	21 x 21	Larger suppresses more noise.
MOG2 history	500 frames	How long the background model remembers.
MOG2 variance threshold	25	Higher means fewer false positives.
Mask threshold	200	Removes shadow pixels.
Morphology kernel	5 x 5 ellipse	Removes speckles, fills small holes.
Minimum contour area	about 1,500 px at 640 wide	Ignores tiny movement.
Confirmation frames	3 consecutive	Requires sustained movement before alerting.
Cooldown	2 seconds	Keeps the alert visible after movement stops.
Warm-up	30 frames	Ignores detections while the background is learned.

9. User interface design

Component	Function	Behaviour
Application title	Displays the application name	Fixed at the top of the window.
Camera feed	Shows live webcam footage	Alert banner is drawn on the feed itself.
Motion status	Shows current detection status	"No Motion Detected" or "Motion Detected!".
Status indicator	Colour cue	Grey when idle, green when monitoring normally, red during motion.
Start Monitoring	Starts camera and detection	Disabled while monitoring.
Stop Monitoring	Stops detection and releases the camera	Disabled while idle.
Error message	Displays camera-related errors	Plain-language text with a suggested fix.

The interface should be simple, modern and easy for a non-technical client to operate.

10. Testing plan

#	Test case	Expected result	Phase
1	Launch application	Application opens successfully.	5
2	Camera available	Live feed appears.	2
3	Camera unavailable or in use by another app	Appropriate error appears; no crash.	2
4	Camera access denied in Windows settings	Error explains how to allow access.	2
5	Person moves in view	Motion alert appears, status turns red.	3, 4
6	No movement	"No Motion Detected", status green.	3, 4
7	Small background changes (curtain, light flicker)	Excessive false alerts are avoided.	3, 7
8	Lighting switched on or off	At most a brief alert, then stable after the background adapts.	7
9	Continuous movement	Banner stays steady, no flicker.	4
10	Stop monitoring	Camera is released; camera light goes off.	6
11	Restart monitoring	Camera works again.	6
12	Close window while running	All camera resources released, process exits.	6
13	Check storage	No video or image files are saved.	6, 7
14	Check network activity	No connections are made.	7
15	Disconnect camera while running	Error shown, camera released, application stays open.	7
16	50 repeated start and stop cycles	No memory growth, no stuck camera.	7
17	Run for one hour or longer	Stable CPU and memory, interface responsive.	7
18	Run the packaged .exe on a clean machine	All of the above work without Python installed.	8

11. Privacy and security

Privacy is particularly important because the application uses a camera.

- The application does not record video and does not save frames as images.
- No video or image data is transmitted over the internet; no networking library is used.
- Frames exist in memory only and are overwritten on each cycle.
- The camera is accessed only while monitoring is active, and is released on Stop or when the application closes.
- Logs contain events and errors only, never image data.
- Privacy is verified, not assumed: a code search in Phase 6, and file-monitor and network checks in Phase 7 (tests 13 and 14).

12. Timeline

The estimates assume the webcam and development environment work correctly. Tasks run one after another, as each phase depends on the previous one.

Phase	Estimate	Schedule (bar = best case, light = extra in worst case; one tick = 0.5 day)
1. Environment setup	0.5 day	
2. Camera integration	0.5 to 1 day	
3. Motion detection	1 to 2 days	
4. Alert system	0.5 day	
5. GUI development	1 to 2 days	
6. Integration and privacy	0.5 day	
7. Testing and debugging	1 to 2 days	
8. Packaging and delivery	0.5 to 1 day	
Total	5.5 to 9.5 days	Best case 5.5 working days; worst case 9.5 working days.

The bars show the sequence and relative size of each phase, not calendar dates. Phase 4 can overlap with the start of Phase 5, which could shorten the total slightly.

Milestones

ID	Milestone	Reached at end of	Evidence
M1	Environment ready	Phase 1	Webcam check script runs.
M2	Live feed and error handling	Phase 2	Feed shown; errors handled.
M3	Detection and alerts working (command-line demo)	Phase 4	Movement triggers a stable alert on the feed.
M4	Complete application running from source	Phase 6	End-to-end run with the GUI, privacy review passed.
M5	Tested and stable	Phase 7	All test cases pass.
M6	Delivered	Phase 8	Executable confirmed by the client.

13. Risks and mitigations

Risk	Impact	Mitigation
Lighting at the client's site differs from the test setup	False or missed alerts	All parameters in <code>config.py</code> ; tune in the client's real environment; keep min-area and variance threshold easy to adjust.
Webcam auto-exposure or auto-focus changes the whole image	False alerts	Warm-up period, shadow removal, and ignoring frames where a very large share of pixels change.
Camera is in use by another application, or blocked by Windows privacy settings	Cannot start	Specific error messages telling the user what to check.
Interface freezes during processing	Poor usability	Worker thread plus queue of size 1; only the main thread updates widgets.
Camera stays locked after a crash or forced close	Camera unusable until restart	Release in a <code>finally</code> block; handle the window-close event; daemon worker thread.
Packaged executable fails on the client's machine	Delivery delay	Test on a clean machine early in Phase 8; keep the dependency list small.
Antivirus flags a one-file PyInstaller executable	Client cannot run it	Test with the client's antivirus; provide the folder-based build as a fallback.
Requirements change during development (for example sound or email alerts)	Schedule and scope	Agree changes in writing; sound fits the <code>on_alert</code> hook; remote messages need a separate design because they break the offline rule.

14. Assumptions and items to confirm with the client

Assumptions: Windows 10 or 11; a working built-in or USB webcam; one camera; one user; the client can test on their own machine at the end of Phase 8.

To confirm before development starts:

1. Is Windows the only target platform?
2. Does the client use a built-in or an external USB webcam?
3. Should the alert be text on the feed only, or also a sound?
4. Should very small movements be ignored, and how large a movement should count?
5. Is it confirmed that all processing stays on the client's computer and nothing is sent anywhere?

15. Development guidelines

- Each feature is developed and tested on its own before it is combined with others.
- Code is modular, with one clear responsibility per file, and the main application contains no experimental code.
- Camera processing and GUI updates are kept apart so the interface does not freeze.
- Every phase is tested before the next one starts.
- Privacy requirements are verified in testing, not assumed.
- Work is committed to Git at the end of each task, with a tag at each milestone.
- Parameters are never hard-coded outside `config.py`.
- Errors are caught, shown in plain language and logged without image data.

Optional extensions (after version 1)

Sound alert, adjustable sensitivity slider, region-of-interest selection, a motion-event counter (count only, no images), multiple cameras, and an optional notification channel if the client approves leaving the offline design.

16. Final deliverables

- Fully functional motion detection desktop application.
- Real-time webcam monitoring.
- Motion detection and visual alert functionality.
- Windows executable (`.exe`).
- Application source code.
- Requirements file.
- README documentation.
- Basic user instructions.
- Test results summary covering the test cases in section 10.

Definition of done

The project is complete when every phase's acceptance criteria are met, all test cases pass (including the no-storage and no-network checks), the executable runs on a clean Windows machine, and the client has received the full deliverables list above and confirmed the application works with their webcam.